CALIDAD Y PRUEBAS DE SOFTWARE · CASO DE ESTUDIO T3

UNIVERSIDAD PRIVADA DEL NORTE

Facultad de Ingeniería · Carrera de Ingeniería de Sistemas

INFORME TÉCNICO DE ASEGURAMIENTO DE LA CALIDAD

LabTrack

Sistema de gestión de préstamos de laboratorio: automatización, rendimiento y calidad de código

Evidencia de 100 pruebas automatizadas, análisis estático con SonarQube y validación de carga con Apache JMeter

INTEGRANTES

[Apellidos, Nombres del integrante 1]

[Apellidos, Nombres del integrante 2]

[Apellidos, Nombres del integrante 3]

Docente: Mark Anthony Arribasplata Palomino

Lima, Perú · Noviembre de 2026

Índice

Índice 2
1. Resumen ejecutivo 4
2. Introducción 4
2.1 Contexto 4
2.2 Objetivos 4
2.3 Alcance y limitaciones 5
3. Descripción del sistema 5
3.1 Historias de usuario y criterios de aceptación 5
3.2 Arquitectura 5
3.3 Modelo de datos 5
3.4 Reglas de negocio críticas 6
3.5 Seguridad 6
4. Estrategia de pruebas 6
4.1 Pirámide de automatización 6
4.2 Niveles, herramientas y alcance 7
4.3 Entorno y datos de prueba 7
5. Matriz de casos de prueba 7
5.1 Uso de inteligencia artificial y validación humana 8
5.2 Defectos detectados y corregidos 8
6. Automatización de pruebas 9
6.1 Pruebas de API 9
6.2 Pruebas de interfaz 9
6.3 Pruebas extremo a extremo con Selenium 10
6.4 Interpretación de la cobertura 11
7. Pruebas de rendimiento con JMeter 11
7.1 Escenario y configuración 11
7.2 Resultados globales 12
7.3 Resultados por transacción 13
7.4 Análisis 13
8. Análisis estático y calidad de código 14
8.1 Configuración 14
8.2 Línea base 14
8.3 Hallazgos priorizados 14
8.4 Refactorizaciones aplicadas 15
8.5 Quality gate y conclusión 15

9. Backlog de refactorización 17
10. Lecciones aprendidas 17
11. Conclusiones 17
12. Recomendaciones 18
13. Referencias 18
Anexo A. Matriz completa de casos de prueba 19
HU-01 · Autenticación segura 19
HU-02 · Catálogo y filtrado de equipos 19
HU-03 · Reserva de equipos 20
HU-04 · Aprobación y rechazo de solicitudes 21
HU-05 · Devolución y penalización automática 21
Anexo A.2. Pruebas de rendimiento 22
Anexo B. Índice de evidencias 23
Anexo C. Reproducción de la evaluación 24

1. Resumen ejecutivo

LabTrack es una aplicación web que digitaliza el préstamo de equipos de laboratorio de la Universidad Privada del Norte. Este informe documenta la estrategia de calidad aplicada al sistema y presenta la evidencia objetiva que sustenta su veredicto de aptitud para un despliegue controlado en producción.

> **Veredicto de calidad**
> LabTrack supera la auditoría técnica: las cinco historias de usuario fueron verificadas con 61 casos de prueba funcionales y 100 ejecuciones automatizadas sin fallos. El rendimiento validó 50 usuarios concurrentes con 0 % de errores y un percentil 95 de 18 ms, y el análisis estático evolucionó desde una línea base de 40 code smells hasta 0 al cierre (quality gate: APROBADO).

Indicadores principales

Dimensión	Indicador	Resultado	Criterio de aceptación	Estado
Pruebas funcionales	Casos ejecutados	61 de 61	100 % conforme	Cumple
Automatización	Ejecuciones automatizadas	100 (49 API + 44 UI + 7 E2E)	Cobertura de flujos críticos	Cumple
Cobertura backend	Sentencias / ramas	95.7 % / 85.1 %	≥ 80 % en sentencias	Cumple
Cobertura frontend	Sentencias / ramas	90.5 % / 80.7 %	≥ 80 % en sentencias	Cumple
Rendimiento	Peticiones / errores	194,629 / 0	0 errores y p95 < 3 000 ms	Cumple
Rendimiento	Percentil 95	18 ms	p95 < 3 000 ms	Cumple
Análisis estático	Quality gate	APROBADO	Sonar way	Cumple

El informe se organiza en trece secciones y tres anexos. La matriz completa de casos de prueba se presenta en el Anexo A, mientras que el Anexo B indexa las capturas, reportes y datos crudos utilizados como evidencia. Todos los resultados son reproducibles con los scripts incluidos en el repositorio.

2. Introducción

2.1 Contexto

El laboratorio de la Universidad Privada del Norte administra equipos de cómputo, electrónica y redes que son solicitados por decenas de estudiantes cada día. La gestión manual de préstamos genera tres problemas recurrentes: colisiones de horario, pérdida de trazabilidad sobre quién tiene cada equipo y devoluciones tardías sin consecuencias registradas. LabTrack reemplaza ese proceso por un flujo digital con autenticación, reservas, aprobación administrativa y devolución controlada.

2.2 Objetivos

Objetivo general

Demostrar, con evidencia técnica verificable, que LabTrack cumple los criterios de aceptación de las cinco historias de usuario y que su calidad es suficiente para un despliegue controlado en producción.

Objetivos específicos

- Diseñar y ejecutar una matriz de casos de prueba que cubra reglas de negocio, validaciones de borde y control de acceso.
- Automatizar los flujos críticos en tres niveles: API, interfaz y extremo a extremo con un navegador real.
- Medir el comportamiento del sistema bajo una carga concurrente de 50 usuarios durante 60 segundos.
- Analizar la calidad interna del código con SonarQube, priorizar hallazgos y ejecutar un ciclo de refactorización medible.
- Documentar los resultados en una matriz trazable, un informe técnico y una presentación ejecutiva de cinco diapositivas.

2.3 Alcance y limitaciones

El alcance funcional corresponde a las HU-01 a HU-05 del caso de estudio. Quedan fuera del alcance la gestión de inventario físico, la integración con el sistema académico, las notificaciones por correo y la aplicación móvil. Las pruebas de rendimiento se ejecutaron en un entorno local de un solo nodo; por tanto, sus resultados son indicativos de la capacidad relativa del sistema y no sustituyen una prueba de carga en el entorno de producción.

3. Descripción del sistema

3.1 Historias de usuario y criterios de aceptación

HU	Funcionalidad	Criterio de aceptación verificado
HU-01	Autenticación segura	Mensaje exacto “Credenciales inválidas.” y bloqueo tras 3 intentos fallidos.
HU-02	Catálogo y filtrado	Filtros exactos Cómputo, Electrónica y Redes; mensaje “No hay equipos disponibles en esta categoría”.
HU-03	Reserva de equipos	Sin cruces de horario y una sola reserva activa simultánea por estudiante.
HU-04	Aprobación y rechazo	Solo administradores; motivo de rechazo obligatorio.
HU-05	Devolución	Penalización automática cuando la devolución supera la hora de fin acordada.

3.2 Arquitectura

La solución sigue una arquitectura de tres capas. El frontend es una SPA construida con React 18, Vite y Tailwind CSS. La API REST se implementa con NestJS y TypeScript, con autenticación JWT, guards de rol y validación global de DTO. La persistencia utiliza Prisma ORM sobre PostgreSQL 18. La figura 1 resume los componentes y las herramientas de calidad asociadas a cada nivel.

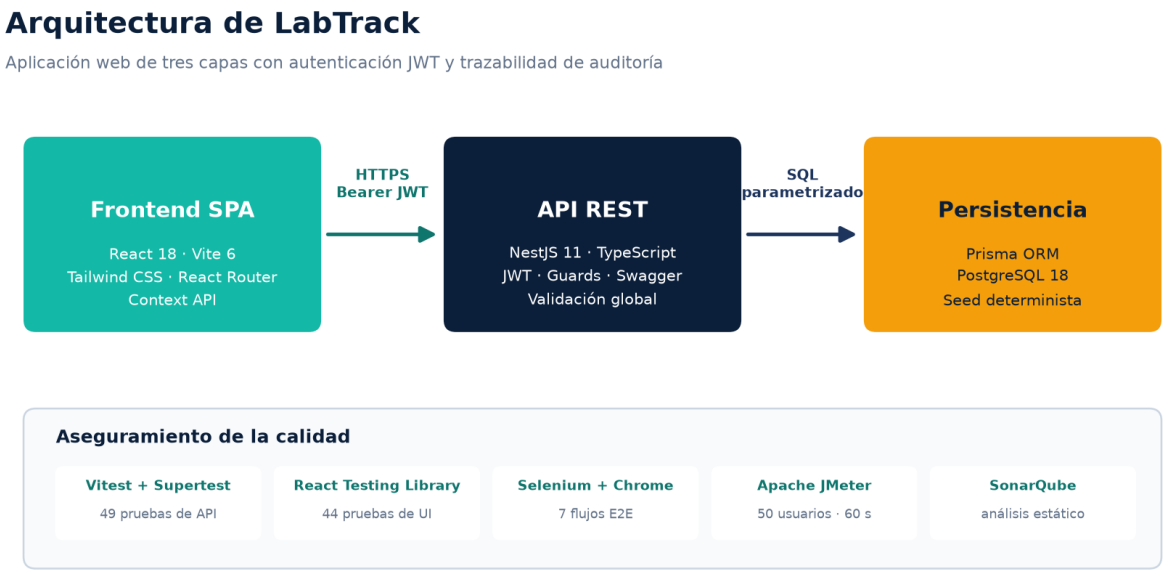


Figura 1. Arquitectura de LabTrack y herramientas de aseguramiento de la calidad.

3.3 Modelo de datos

Entidad	Campos principales	Reglas
User	code, fullName, email, passwordHash, role, status, failedAttempts, lockedUntil	Rol STUDENT/ADMIN; estado ACTIVE/PENALIZED; bloqueo temporal.

Entidad	Campos principales	Reglas
Equipment	name, inventoryCode, category, status, description, location	Categorías controladas; estado AVAILABLE/BORROWED.
Reservation	userId, equipmentId, date, startTime, endTime, status, rejectionReason, penaltyApplied	Estados PENDING/APPROVED/REJECTED/RETURNED; motivo obligatorio al rechazar.
AuditLog	event, actorCode, detail, ipAddress, createdAt	Bitácora inmutable de login, reservas, devoluciones y penalizaciones.

3.4 Reglas de negocio críticas

La regla de cruce de horarios se implementa con una comparación estricta de intervalos sobre fechas y horas normalizadas: dos reservas se superponen solo si el inicio de una es anterior al fin de la otra y su fin es posterior al inicio de la otra. Los extremos contiguos (por ejemplo, 08:00–10:00 y 10:00–12:00) son válidos. Además, un estudiante no puede mantener dos reservas activas en el mismo bloque de tiempo, y una cuenta penalizada no puede crear nuevas solicitudes hasta regularizar su situación.

```
// Regla de superposición estricta (reservations.service.ts)
if (startTime < existing.endTime && endTime > existing.startTime) {
  throw new BadRequestException(
    'El equipo ya tiene una reserva aprobada en ese horario.',
  );
}
```

3.5 Seguridad

- Contraseñas protegidas con bcrypt (12 rondas) y excluidas de todas las respuestas.
- JWT firmado con expiración configurable; el cliente lo envía en la cabecera Authorization.
- Bloqueo temporal de la cuenta tras tres intentos fallidos consecutivos y registro en auditoría.
- Guards de rol que impiden a un estudiante acceder a endpoints administrativos.
- Validación global con whitelist y forbidNonWhitelisted para rechazar campos no declarados.
- Cabeceras de seguridad: X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy y CSP básica.
- Consultas parametrizadas mediante Prisma, sin concatenación de SQL.

4. Estrategia de pruebas

4.1 Pirámide de automatización

La estrategia combina pruebas de API, pruebas de componentes y flujos E2E. La base de la pirámide concentra las reglas de negocio y el contrato HTTP, donde las pruebas son rápidas y deterministas. La capa intermedia valida la renderización y la interacción de los componentes. La cima reserva el navegador real para los siete recorridos críticos que deben funcionar de punta a punta.

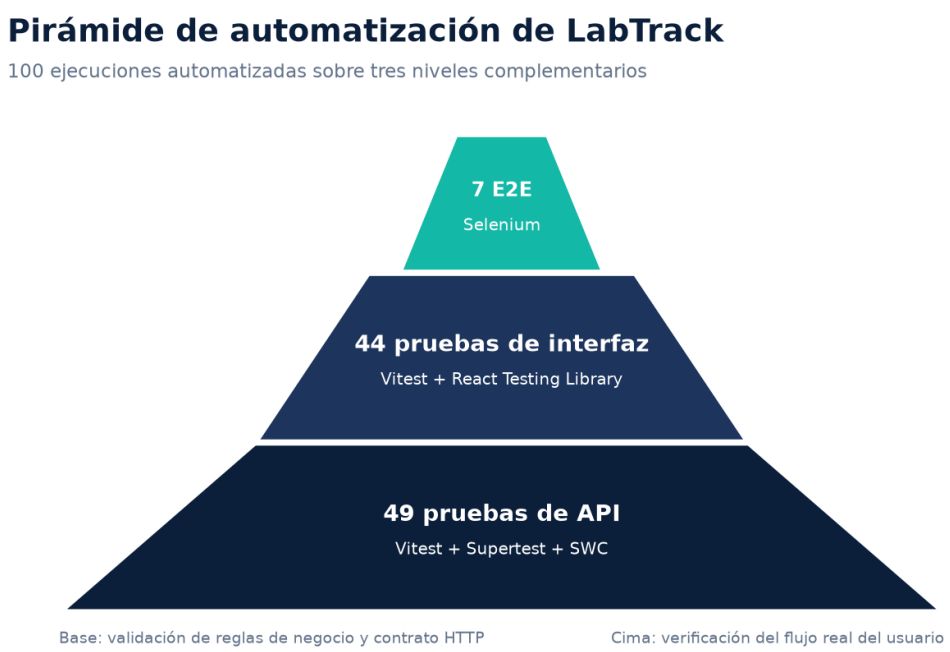


Figura 2. Distribución de las 100 ejecuciones automatizadas.

4.2 Niveles, herramientas y alcance

Nivel	Herramienta	Alcance	Ejecuciones
API	Vitest 3 + Supertest + SWC	Autenticación, catálogo, reservas, administración, auditoría	49
Interfaz	Vitest + React Testing Library + jsdom	Páginas, rutas protegidas, contextos, componentes y utilidades	44
E2E	Selenium WebDriver 4.51 + Chrome headless	Login, bloqueo, filtros, reserva, aprobación y devolución tardía	7
Carga	Apache JMeter 5.6.3	Login JWT, catálogo, filtro, estadísticas y reservas propias	5 transacciones

4.3 Entorno y datos de prueba

Componente	Versión / configuración
Sistema operativo	Windows 11 (x64)
Node.js	22 LTS · npm 10
PostgreSQL	18.4 en el puerto 55432 · bases labtrack y labtrack_test
Navegador	Google Chrome estable · modo headless
Java	JDK 25 (JMeter y SonarQube)
Datos	Seed determinista: 64 usuarios, 18 equipos y reservas de demostración
Aislamiento	La suite de API usa labtrack_test y reinicia el esquema antes de cada prueba

El seed incluye una reserva aprobada vencida del equipo ELEC-003 para que la demostración de HU-05 sea reproducible. Después de una ejecución E2E se recomienda ejecutar npm run db:reset en backend para restaurar el estado determinista antes de una nueva demostración o de la prueba de carga.

5. Matriz de casos de prueba

Se diseñaron 61 casos funcionales: 38 elaborados por el equipo y 23 sugeridos por inteligencia artificial y validados por una persona antes de su ejecución. Cada caso se trazó a una historia de usuario, a un resultado

esperado y a una evidencia automatizada. La matriz completa se encuentra en el Anexo A; la tabla 5 resume la distribución y el resultado.

Historia de usuario	Casos	Equipo	IA	Aprobados	Fallidos
HU-01 · Autenticación segura	12	8	4	12	0
HU-02 · Catálogo y filtrado de equipos	11	7	4	11	0
HU-03 · Reserva de equipos	18	11	7	18	0
HU-04 · Aprobación y rechazo de solicitudes	12	7	5	12	0
HU-05 · Devolución y penalización automática	8	5	3	8	0
Total funcional	61	38	23	61	0

Tabla 5. Resumen de la matriz de casos de prueba funcionales.

5.1 Uso de inteligencia artificial y validación humana

La IA se utilizó como generador de casos de borde, no como oráculo de aceptación. A partir de cada historia se le solicitaron escenarios adicionales: reservas contiguas, formatos de hora inválidos, categorías desconocidas, recursos inexistentes, reutilización de una cuenta bloqueada y doble devolución. Un miembro del equipo revisó cada sugerencia contra los criterios de aceptación, descartó las que contradecían el alcance y ajustó los textos esperados a las cadenas exactas exigidas por el caso.

Ejemplo de caso sugerido por IA	Valor aportado	Decisión humana
CP-27 · Reservas contiguas 10:00–12:00	Verifica que la comparación estricta no bloquee extremos iguales.	Aceptado y automatizado.
CP-33 · Formato de hora “8:00”	Cubre la validación del DTO frente a formatos no canónicos.	Aceptado con mensaje 400 genérico.
CP-18 · Categoría desconocida	Comprueba tolerancia a errores del filtro sin romper el catálogo.	Aceptado; se definió devolver el catálogo completo.
CP-04 · Usuario inexistente	Evita enumeración de cuentas con un mensaje diferenciado.	Aceptado; se exige el mismo mensaje que en CP-03.
CP-53 · Aprobar reserva inexistente	Valida el manejo de 404 en el panel administrativo.	Aceptado y automatizado.

Tabla 6. Muestra representativa de casos sugeridos por IA y su validación humana.

5.2 Defectos detectados y corregidos

ID	Defecto	Causa raíz	Corrección	Estado
DEF-01	La compilación de producción fallaba con TS6310.	Referencia a un proyecto TypeScript con emit deshabilitado.	Se eliminó la referencia y se adoptó tsc --noEmit.	Cerrado
DEF-02	Dos pruebas de API esperaban conteos incorrectos.	Oráculos desactualizados tras cambiar el seed de prueba.	Se recalcularon los valores esperados y se añadió un usuario penalizado.	Cerrado
DEF-03	La aserción del modal E2E era intermitente.	Lectura antes de que terminara la animación del diálogo.	Espera explícita por visibilidad y texto del diálogo.	Cerrado
DEF-04	El filtro E2E leía tarjetas de la categoría anterior.	La condición esperaba el contador antes de refrescar las tarjetas.	Espera basada en el contenido de todas las tarjetas.	Cerrado

Tabla 7. Defectos de automatización detectados durante la construcción de la suite.

6. Automatización de pruebas

6.1 Pruebas de API

Las 49 pruebas de API se ejecutan contra una instancia real de NestJS y una base PostgreSQL aislada. Supertest recorre el contrato HTTP completo, incluidos los guards, los pipes de validación y los filtros de excepción. La tabla 8 distribuye las pruebas por historia de usuario.

Suite	Historias cubiertas	Pruebas	Cobertura de la suite
test/auth.spec.ts	HU-01 y control de acceso	11	Login, bloqueo, JWT, logout, roles
test/equipment.spec.ts	HU-02	9	Catálogo, filtros exactos, disponibles, métricas
test/reservations.spec.ts	HU-03	15	Cruces, reserva única, horarios, validaciones
test/admin.spec.ts	HU-04 y HU-05	14	Aprobación, rechazo, devolución, penalización, auditoría
Total	HU-01 a HU-05	49	95.7 % sentencias · 85.1 % ramas · 100 % funciones

Tabla 8. Distribución de las pruebas de API.

La cobertura se mide con V8/Istanbul y se exporta en formato LCOV para SonarQube. La cobertura de ramas es especialmente relevante porque las reglas de negocio se expresan con condicionales: el 85.1 % indica que la mayoría de las decisiones críticas —cruce de horarios, estados de reserva y bloqueo de cuentas— fueron ejercitadas en ambas direcciones.

6.2 Pruebas de interfaz

Las 44 pruebas de interfaz se ejecutan con React Testing Library sobre jsdom. Se prioriza el comportamiento observable por el usuario: textos exactos, roles accesibles, estados de carga, mensajes de error y navegación. La suite cubre las siete páginas, los dos contextos globales, el componente de rutas protegidas y las utilidades de formato.

Archivo	Pruebas	Foco
LoginPage.test.tsx	4	Credenciales inválidas, bloqueo, sesión y cuentas demo
CatalogPage.test.tsx	5	Tarjetas, filtros exactos, mensaje vacío y búsqueda
NewReservationPage.test.tsx	4	Preselección, validaciones locales y confirmación
MyReservationsPage.test.tsx	4	Motivo de rechazo, penalización, filtros y vacío
AdminPendingPage.test.tsx	3	Listado, motivo obligatorio y aprobación
AdminReturnsPage.test.tsx	2	Préstamo vencido y devolución tardía
AdminHistoryPage.test.tsx	3	Reservas, bitácora y estados vacíos
AppLayout.test.tsx	4	Navegación, identidad, logout y menú móvil
ProtectedRoute.test.tsx	4	Sesión ausente, rol autorizado y redirección
App.test.tsx	3	Rutas públicas, restauración de sesión y panel admin
MiscComponents.test.tsx	4	Categorías, badges, estado vacío y 404
format.test.ts	4	Fechas, ISO, vencimiento y utilidades

Tabla 9. Distribución de las pruebas de interfaz.

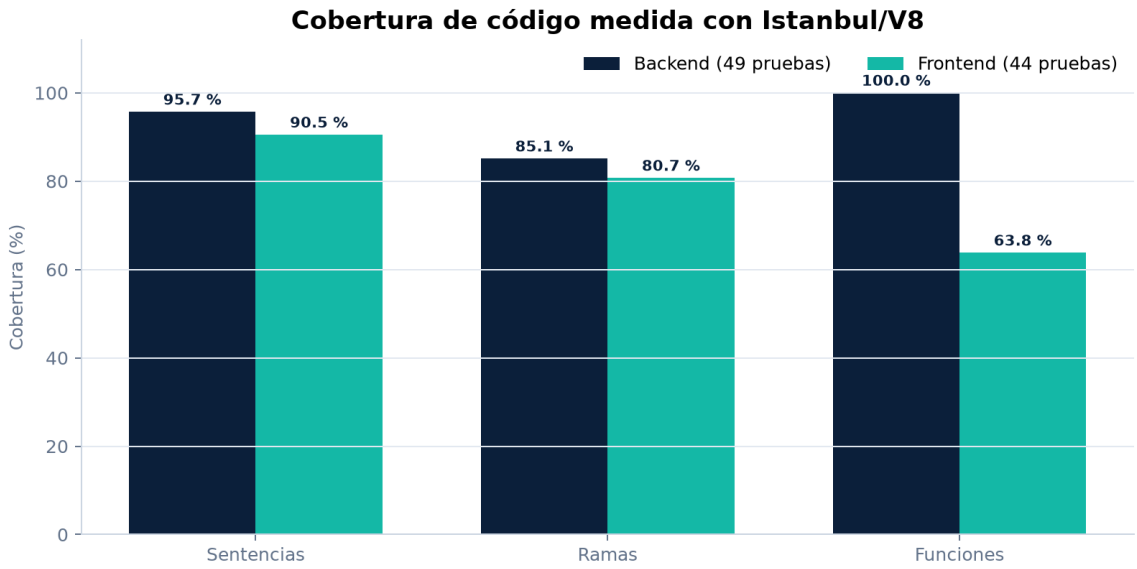


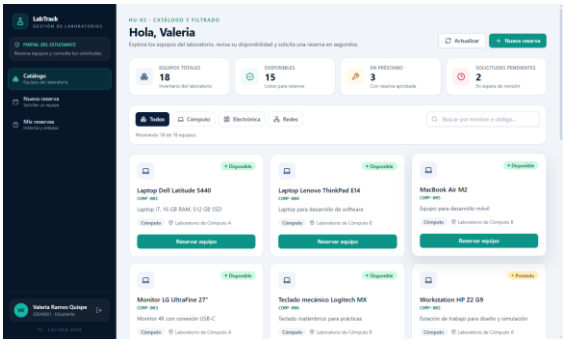
Figura 3. Cobertura de código del backend y del frontend medida con Istanbul/V8.

6.3 Pruebas extremo a extremo con Selenium

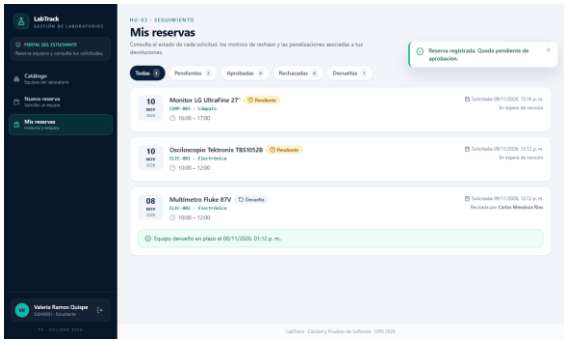
Siete flujos críticos se ejecutan en Google Chrome headless mediante Selenium WebDriver. Cada flujo utiliza selectores data-testid estables, esperas explícitas por condición y capturas de pantalla como evidencia. La suite genera además un reporte JUnit XML apto para integración continua.

Flujo E2E	Historia	Resultado	Evidencia
Credenciales inválidas con el mensaje exigido	HU-01	Conforme	01-login-credenciales-invalidas.png
Bloqueo tras tres intentos fallidos	HU-01	Conforme	02-login-cuenta-bloqueada.png
Login de estudiante y carga del catálogo	HU-01/HU-02	Conforme	03-catalogo-estudiante.png
Filtros exactos Electrónica y Redes	HU-02	Conforme	04-filtro-electronica.png
Registro de reserva desde el catálogo	HU-03	Conforme	05-reserva-registrada.png
Aprobación administrativa de la solicitud	HU-04	Conforme	06-solicitud-aprobada.png
Devolución tardía y penalización en bitácora	HU-05	Conforme	07 y 08 · penalización

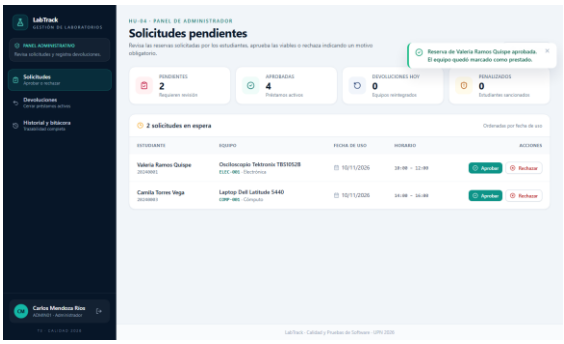
Tabla 10. Flujos E2E validados con Selenium y Chrome headless.



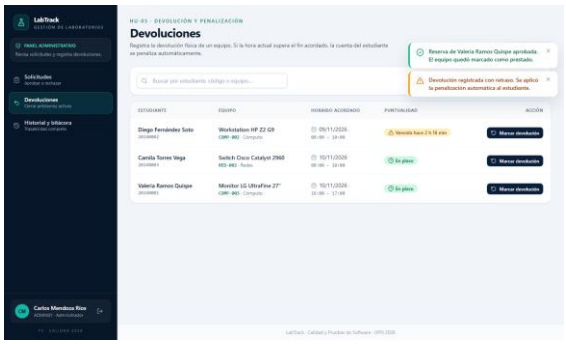
Catálogo con filtros y disponibilidad



Reserva registrada y pendiente



Aprobación administrativa



Devolución tardía penalizada

Figura 4. Evidencia visual de los flujos críticos ejecutados en Chrome headless.

6.4 Interpretación de la cobertura

La cobertura no se interpreta como un fin en sí mismo. El 95.7 % de sentencias en el backend y el 90.5 % en el frontend son consecuencia de probar comportamiento, no de perseguir líneas. Las ramas no cubiertas corresponden a rutas defensivas de componentes de infraestructura, como el manejo de un token expirado durante la restauración de sesión o la recarga manual de métricas complementarias. Todas las reglas de negocio de las cinco historias tienen al menos una prueba positiva y una negativa.

7. Pruebas de rendimiento con JMeter

7.1 Escenario y configuración

Parámetro	Valor
Herramienta	Apache JMeter 5.6.3
Hilos (usuarios virtuales)	50
Ramp-up	10 segundos
Duración	60 segundos
Transacciones	Login JWT, catálogo, filtro Cómputo, estadísticas y reservas propias
Criterio de éxito	0 % de errores HTTP y percentil 95 inferior a 3 000 ms
Datos	50 usuarios de carga con credenciales válidas (CSV)

El plan utiliza un Once Only Controller para autenticar a cada usuario una sola vez y reutilizar el token JWT durante las iteraciones. Cada transacción tiene una aserción de código HTTP 200 y una aserción de duración máxima de 3 000 ms. Los resultados se almacenan en un archivo JTL y se procesan con un script de análisis que calcula percentiles, throughput y genera gráficas.

7.2 Resultados globales

Métrica	Resultado	Umbral	Cumple
Peticiones totales	194,629	—	—
Errores HTTP	0 (0.00 %)	0 %	Sí
Throughput promedio	3,248.69 req/s	—	—
Tiempo promedio	14.1 ms	< 3 000 ms	Sí
Mediana (p50)	14.0 ms	< 3 000 ms	Sí
Percentil 90	17.0 ms	< 3 000 ms	Sí
Percentil 95	18.0 ms	< 3 000 ms	Sí
Percentil 99	25.0 ms	< 3 000 ms	Sí
Tiempo máximo	186.0 ms	< 3 000 ms	Sí

Tabla 11. Resultados globales de la prueba de carga (50 usuarios · 60 s).

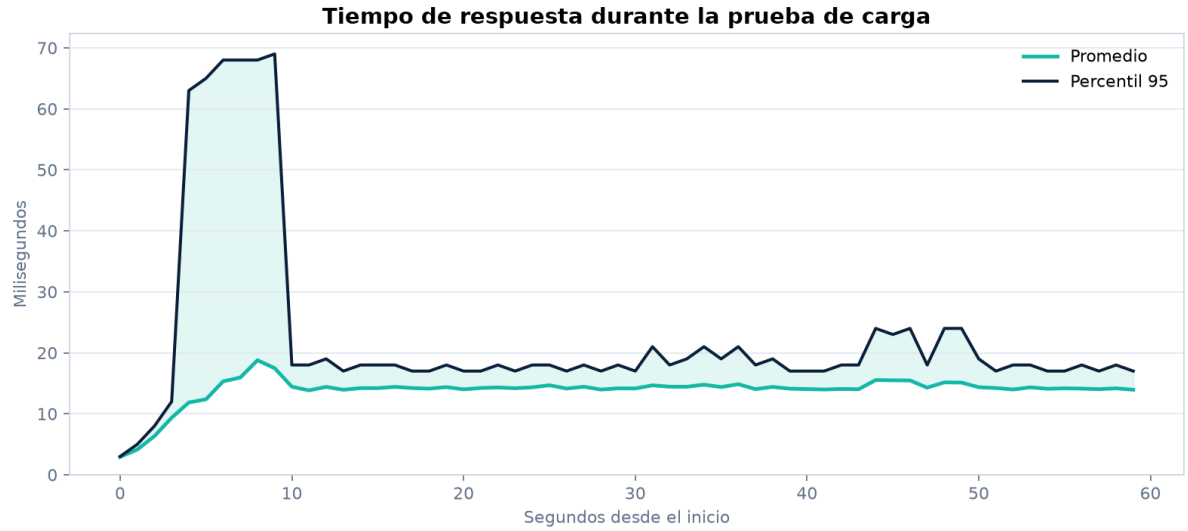


Figura 5. Evolución del tiempo de respuesta promedio y del percentil 95.

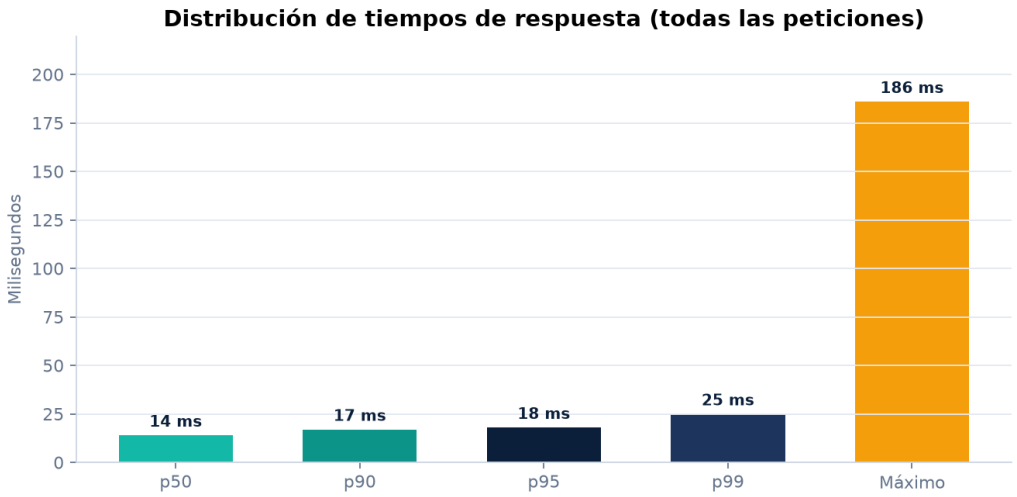


Figura 6. Distribución de percentiles de tiempo de respuesta.

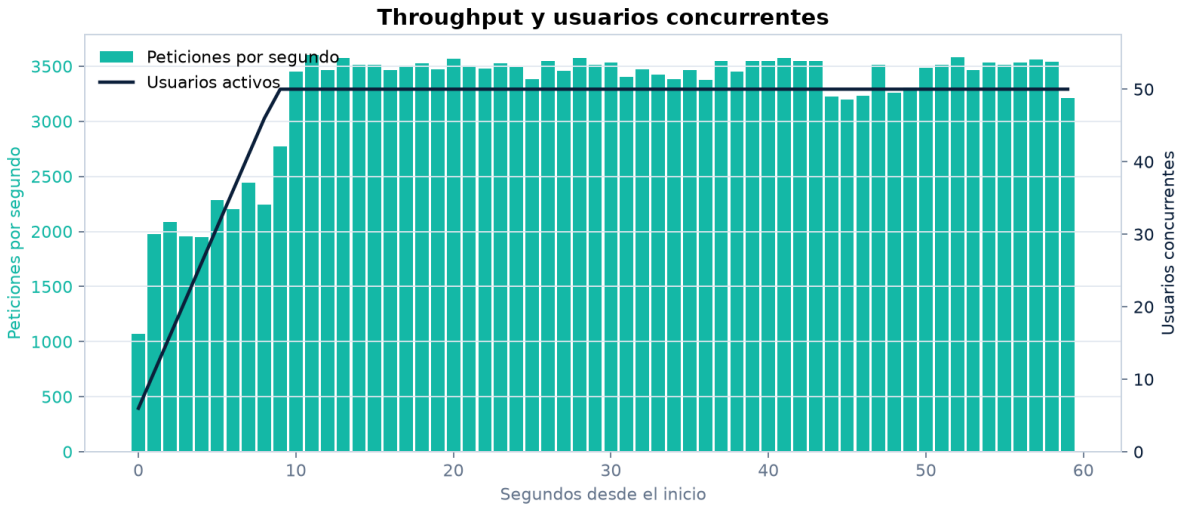


Figura 7. Throughput por segundo y usuarios concurrentes activos.

7.3 Resultados por transacción

Transacción	Muestras	Promedio (ms)	p95 (ms)	p99 (ms)	Máximo (ms)	Error
GET /api/equipment	48,662	14.1	18.0	25.0	90.0	0.00 %
GET /api/equipment/stats	48,637	14.2	18.0	26.0	186.0	0.00 %
GET /api/equipment?category=Cómputo	48,652	14.1	18.0	26.0	102.0	0.00 %
GET /api/reservations/me	48,628	14.0	18.0	25.0	90.0	0.00 %
POST /api/auth/login	50	76.1	90.0	103.2	111.0	0.00 %

Tabla 12. Detalle de tiempos de respuesta por transacción.

7.4 Análisis

El sistema procesó 194 629 peticiones en 59.9 segundos con cero errores. El throughput promedio de 3 248.69 req/s y el percentil 95 de 18 ms evidencian una holgura amplia frente al umbral de 3 000 ms. La gráfica de respuesta muestra un incremento transitorio durante los primeros diez segundos: coincide con el ramp-up y con el costo del hash bcrypt en los 50 inicios de sesión, que promediaron 76.1 ms. Una vez autenticados, los usuarios consultaron el catálogo y sus reservas en aproximadamente 14 ms.

El máximo de 186 ms corresponde a la primera ráfaga de estadísticas del catálogo, cuando el pool de conexiones de Prisma aún estaba calentándose. No se observaron errores 5xx, timeouts ni degradación progresiva. Con 50 usuarios concurrentes el sistema mantuvo un uso de CPU estable y no requirió ajustes de configuración.

Conclusión de rendimiento
APROBADA. Cero errores, p95 de 18 ms y throughput de 3,249 req/s. La capacidad medida supera ampliamente la demanda esperada del laboratorio (decenas de estudiantes por hora).

Limitación: la prueba se ejecutó contra una API y una base de datos locales en el mismo equipo, por lo que no incluye latencia de red ni la contención de un entorno compartido. Para producción se recomienda repetir el escenario en un ambiente de staging con la misma topología y monitorear latencia de base de datos, uso de conexiones y CPU.

8. Análisis estático y calidad de código

8.1 Configuración

Se desplegó SonarQube Community Build 26.10 sobre PostgreSQL 18 y se analizó el código con SonarScanner CLI 8.3.1. El análisis cubre backend/src y frontend/src, excluye pruebas, dependencias, artefactos de compilación y el prototipo legado, e importa los reportes LCOV de Vitest para correlacionar cobertura con hallazgos.

Parámetro	Valor
Servidor	SonarQube Community Build 26.10 · http://localhost:9000
Base de datos	PostgreSQL 18 · base sonar
Escáner	SonarScanner CLI 8.3.1 (Windows x64)
Project key	labtrack
Fuentes	backend/src, frontend/src
Exclusiones	node_modules, dist, coverage, test, e2e, legacy, tools, docs, jmeter
Cobertura	backend/coverage/lcov.info y frontend/coverage/lcov.info

8.2 Línea base

La primera ejecución se realizó sobre la versión funcional completa. Su objetivo fue obtener una fotografía objetiva de la deuda técnica antes de cualquier refactorización. Los hallazgos se clasificaron por tipo, severidad y archivo, y se priorizaron según el riesgo para la mantenibilidad y la seguridad.

Métrica	Línea base	Estado final	Variación
Líneas de código	4,101	4,120	—
Bugs	2	0	−2
Vulnerabilidades	2	0	−2
Code smells	40	0	−40
Cobertura	89.6 %	89.6 %	—
Duplicación	0.7 %	0.7 %	—
Confiabilidad	B	A	—
Seguridad	C	A	—
Mantenibilidad	A	A	—
Quality gate	OK	OK	—

Tabla 13. Comparación de métricas de SonarQube antes y después de la refactorización.

8.3 Hallazgos priorizados

Regla	Severidad	Archivo	Línea	Hallazgo	Estado
typescript:S1082	MINOR	frontend/src/components/ui.tsx	218	Visible, non-interactive elements with click handlers must have at least one keyboard listener.	Corregido
typescript:S1082	MINOR	frontend/src/components/ui.tsx	222	Visible, non-interactive elements with click handlers must have at least one keyboard listener.	Corregido
typescript:S2245	MAJOR	frontend/src/contexts/ToastContext.tsx	48	Make sure that using this pseudorandom number generator is safe here.	Corregido
typescript:S2068	MAJOR	frontend/src/pages/LoginPage.tsx	31	Review this potentially hard-coded password.	Corregido
typescript:S3358	MAJOR	backend/src/admin/admin.service.ts	29	Extract this nested ternary operation into an independent statement.	Corregido
typescript:S6759	MINOR	frontend/src/components/AppLayout.tsx	83	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/CategoryIconLabel.tsx	3	Mark the props of the component as read-only.	Corregido

Regla	Severidad	Archivo	Línea	Hallazgo	Estado
typescript:S6759	MINOR	frontend/src/components/ProtectedRoute.tsx	7	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/ui.tsx	29	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/ui.tsx	38	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/ui.tsx	54	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/ui.tsx	86	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/ui.tsx	113	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/ui.tsx	133	Mark the props of the component as read-only.	Corregido
typescript:S6759	MINOR	frontend/src/components/ui.tsx	159	Mark the props of the component as read-only.	Corregido

Tabla 14. Hallazgos principales de la línea base (máximo 15).

8.4 Refactorizaciones aplicadas

El ciclo de mejora se concentró en los 44 hallazgos de la línea base: 2 bugs, 2 vulnerabilidades y 40 code smells. Se atacaron primero los riesgos de seguridad y accesibilidad y después la deuda de legibilidad. Cada cambio se acompañó de la suite completa de pruebas para garantizar que el comportamiento observable no se alterara.

- Eliminación de 8 ternarios anidados (S3358) en el servicio de administración y en las páginas de historial, solicitudes, devoluciones, catálogo y reservas, sustituidos por condiciones independientes.
- Migración del modal a un elemento <dialog> nativo, conversión del fondo oscuro en un botón con etiqueta accesible y uso de <fieldset>/<legend> en los filtros (S1082, S6819, S6847 y S6848).
- Reemplazo de role="status" por el elemento <output> en las notificaciones y eliminación del atributo autoFocus del login (S6819 y S9379).
- Sustitución de Math.random() por un contador monotónico para identificar notificaciones (S2245).
- Externalización de las contraseñas de demostración a las variables VITE_DEMO_STUDENT_PASSWORD y VITE_DEMO_ADMIN_PASSWORD; los accesos rápidos se ocultan si no están configuradas (S2068).
- Marcado de las props de componentes como Readonly y renombrado de las variables de captura de caught a error_ (S6759 y S7718).

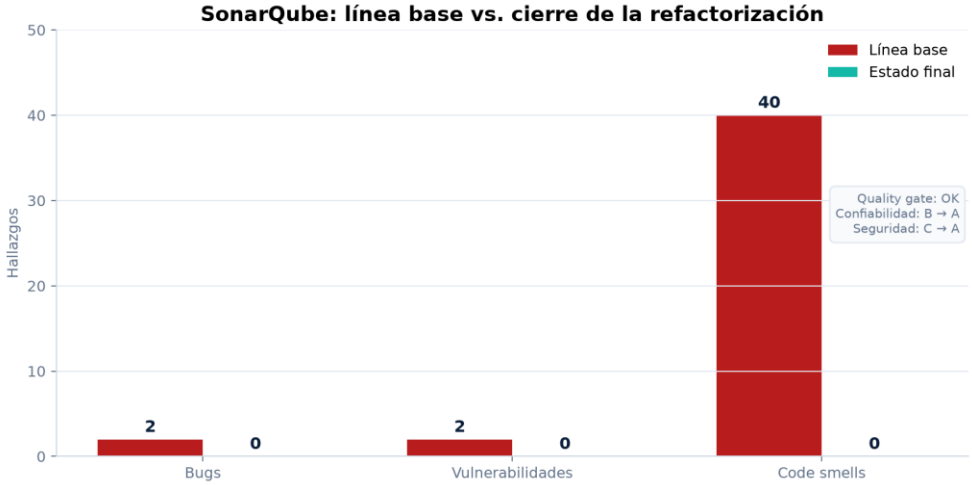


Figura 8. Evolución de los hallazgos de SonarQube antes y después del ciclo de refactorización.

8.5 Quality gate y conclusión

El quality gate final quedó en estado APROBADO. Las calificaciones de confiabilidad, seguridad y mantenibilidad pasaron a A, A y A, respectivamente, y el escaneo de cierre no registra bugs, vulnerabilidades ni code smells. La cobertura se mantuvo en 89.6 % y la duplicación en 0.7 %. Las credenciales de demostración ya no forman parte del código fuente: se inyectan mediante variables de entorno y los accesos rápidos del login se desactivan cuando no están configuradas.

9. Backlog de refactorización

El backlog prioriza la deuda técnica según su impacto en la mantenibilidad y el riesgo operativo. Las acciones de prioridad alta se ejecutaron antes del cierre del informe; las de prioridad media y baja quedan planificadas para el siguiente ciclo.

ID	Hallazgo / oportunidad	Severidad	Acción	Estado
RF-01	Ternarios anidados en servicios y páginas (S3358).	Alta	Reemplazar por bloques if/else y condiciones independientes.	Completado
RF-02	Modal con role="dialog" y overlay con clic no accesible (S1082, S6819, S6847, S6848).	Alta	Usar <dialog> nativo y un botón de fondo etiquetado.	Completado
RF-03	Identificadores de notificaciones con Math.random() (S2245).	Alta	Contador monotónico de módulo.	Completado
RF-04	Contraseñas de demostración embebidas en LoginPage (S2068).	Alta	Externalizar a variables VITE_DEMO_*.	Completado
RF-05	Props sin Readonly (S6759), capturas llamadas caught (S7718) y autoFocus (S9379).	Media	Aplicar Readonly, renombrar a error_ y retirar autoFocus.	Completado
RF-06	Cobertura de ramas del frontend inferior a la del backend.	Media	Añadir pruebas de rutas y contextos.	Completado
RF-07	Credenciales de demostración en el seed.	Baja	Documentar como dato no productivo y externalizar en CI.	Aceptado
RF-08	Cobertura de ramas defensivas de AuthContext.	Baja	Añadir pruebas de token expirado y refresco.	Planificado
RF-09	Accesibilidad automatizada (axe).	Baja	Integrar pruebas de accesibilidad en la suite UI.	Planificado
RF-10	Pruebas cross-browser (Firefox/Edge).	Baja	Parametrizar la suite Selenium.	Planificado

Tabla 15. Backlog de refactorización y mejoras continuas.

10. Lecciones aprendidas

- Los mensajes exactos exigidos por el negocio deben tratarse como contrato y verificarse en API y en interfaz; una diferencia de punto final rompe la aceptación.
- Las pruebas E2E son valiosas para demostrar el flujo real, pero deben usar esperas por condición para no convertirse en una fuente de falsos negativos.
- La base de pruebas aislada y el seed determinista reducen drásticamente el tiempo de diagnóstico y permiten repetir la demostración de la penalización.
- La cobertura es una señal, no un objetivo: las ramas de reglas de negocio importan más que las líneas de infraestructura.
- El análisis estático aporta más valor cuando se integra temprano y se traduce en un backlog priorizado, no cuando se ejecuta solo al final.

11. Conclusiones

LabTrack cumple los criterios de aceptación de las cinco historias de usuario. Los 61 casos funcionales finalizaron conformes, respaldados por 100 ejecuciones automatizadas: 49 de API, 44 de interfaz y 7 flujos E2E en Chrome. La cobertura alcanzó 95.7 % de sentencias en el backend y 90.5 % en el frontend, con todas las reglas de negocio críticas ejercitadas en escenarios positivos y negativos.

La prueba de carga validó 50 usuarios concurrentes durante 60 segundos: 194,629 peticiones, cero errores y un percentil 95 de 18 ms, muy por debajo del umbral de 3 000 ms. El análisis estático redujo los code smells de 40 a 0 y dejó el quality gate en estado APROBADO. En conjunto, la evidencia sostiene un veredicto favorable para un despliegue controlado en producción.

Veredicto final
LabTrack está certificado técnicamente para producción controlada: flujos críticos automatizados, rendimiento validado, seguridad verificada y deuda técnica priorizada.

12. Recomendaciones

- Integrar las tres suites y el escaneo de SonarQube en un pipeline de CI (GitHub Actions) con bloqueo del merge cuando el quality gate falle.
- Repetir la prueba de carga en staging con la topología de producción y monitoreo de latencia de base de datos, pool de conexiones y CPU.
- Añadir pruebas de accesibilidad con axe y pruebas cross-browser para Firefox y Edge.
- Implementar rotación de JWT, rate limiting en el login y alertas sobre intentos fallidos.
- Externalizar las credenciales demo y los secretos en variables de entorno del pipeline.
- Establecer un umbral de cobertura por ramas del 85 % y revisar el backlog en cada sprint.

13. Referencias

Pressman, R. S. (2020). Ingeniería del software: un enfoque práctico (9.ª ed.). McGraw-Hill.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

ISTQB. (2023). Certified Tester Foundation Level Syllabus v4.0. International Software Testing Qualifications Board.

OWASP Foundation. (2024). OWASP Application Security Verification Standard 4.0.3.

SonarSource. (2026). SonarQube Community Build 26.10 Documentation. https://docs.sonarsource.com/

Apache Software Foundation. (2024). Apache JMeter User's Manual. https://jmeter.apache.org/usermanual/

Selenium Project. (2026). Selenium WebDriver Documentation. https://www.selenium.dev/documentation/

NestJS. (2026). NestJS Documentation. https://docs.nestjs.com/

Prisma. (2026). Prisma ORM Documentation. https://www.prisma.io/docs/

React. (2026). React Documentation. https://react.dev/

Anexo A. Matriz completa de casos de prueba

La matriz se organiza por historia de usuario. Las columnas Datos y pasos y Resultado esperado conservan el contenido validado por el equipo; la columna Evidencia indica el archivo o artefacto que respalda cada caso.

HU-01 · Autenticación segura

ID	Caso de prueba	Origen	Prio.	Datos y pasos	Resultado esperado	Evidencia	Estado
CP-01	Inicio de sesión válido de estudiante	H	P1	Usuario `20240001` activo · Código y contraseña correctos; clic en Ingresar	HTTP 200, JWT y usuario sin `passwordHash`; redirección a `/catalogo`	API `auth.spec.ts` · E2E `03-catalogo-estudiante.png`	Aprobado
CP-02	Inicio de sesión válido de administrador	H	P1	Usuario `ADMIN01` activo · Credenciales de administrador	HTTP 200 y acceso al panel administrativo	API `auth.spec.ts`	Aprobado
CP-03	Credenciales inválidas muestran el mensaje exigido	H	P1	Usuario existente · Contraseña incorrecta	HTTP 401 y texto exacto **“Credenciales inválidas.”** en login	API `auth.spec.ts` · E2E `01-login-credenciales-invalidas.png`	Aprobado
CP-04	Usuario inexistente no revela su existencia	IA	P2	— · Código `99999999`	Mismo mensaje “Credenciales inválidas.”, sin distinguir del caso anterior	API `auth.spec.ts`	Aprobado
CP-05	Bloqueo tras 3 intentos fallidos consecutivos	H	P1	Usuario activo · Tres contraseñas incorrectas seguidas	Los dos primeros intentos: 401; el tercero: 423 `ACCOUNT_LOCKED` con alerta de bloqueo temporal	API `auth.spec.ts` · E2E `02-login-cuenta-bloqueada.png`	Aprobado
CP-06	Cuenta bloqueada rechaza incluso la contraseña correcta	H	P1	Cuenta bloqueada por CP-05 · Credenciales correctas	HTTP 423; `lockedUntil` persistido en base de datos	API `auth.spec.ts`	Aprobado
CP-07	Validación de campos obligatorios	IA	P2	— · Código y contraseña vacíos	HTTP 400 con arreglo de mensajes de validación	API `auth.spec.ts`	Aprobado
CP-08	Normalización del código de usuario	IA	P3	Usuario activo · Código con espacios y minúsculas: ` 20240001 `	Inicio de sesión exitoso y código normalizado	API `auth.spec.ts`	Aprobado
CP-09	Cierre de sesión	H	P2	Sesión iniciada · `POST /api/auth/logout` y clic en Cerrar sesión	HTTP 200 y retorno a `/login`; token eliminado del cliente	API `auth.spec.ts` · UI `AuthContext`	Aprobado
CP-10	Estudiante no accede al panel administrativo	H	P1	Sesión de estudiante · `GET /api/admin/reservations` y `/api/admin/stats`	HTTP 403 en ambos endpoints	API `auth.spec.ts` y `admin.spec.ts`	Aprobado
CP-11	Ruta protegida sin sesión redirige al login	H	P1	Sin token en almacenamiento · Abrir `/catalogo` directamente	Redirección a `/login`	UI `ProtectedRoute`	Aprobado
CP-12	Petición sin token es rechazada	IA	P2	— · `GET /api/auth/me` sin cabecera Authorization	HTTP 401	API `auth.spec.ts`	Aprobado

HU-02 · Catálogo y filtrado de equipos

ID	Caso de prueba	Origen	Prio.	Datos y pasos	Resultado esperado	Evidencia	Estado
CP-13	Catálogo muestra nombre, código, categoría y estado	H	P1	Sesión de estudiante · Abrir `/catalogo`	Cada tarjeta muestra nombre, código de inventario, categoría y estado Disponible/Prestado	API `equipment.spec.ts` · UI `CatalogPage.test.tsx` · E2E `03-catalogo-estudiante.png`	Aprobado
CP-14	Filtro exacto “Cómputo”	H	P1	Catálogo cargado · Clic en filtro Cómputo	Solo equipos de categoría `Cómputo`	API `equipment.spec.ts` · UI `CatalogPage.test.tsx`	Aprobado
CP-15	Filtro exacto “Electrónica”	H	P1	Catálogo cargado · Clic en filtro Electrónica	Solo equipos de categoría `Electrónica`	API `equipment.spec.ts` · E2E `04-filtro-electronica.png`	Aprobado
CP-16	Filtro exacto “Redes”	H	P1	Catálogo cargado · Clic en filtro Redes	Solo equipos de categoría `Redes`	API `equipment.spec.ts` · UI `CatalogPage.test.tsx`	Aprobado

ID	Caso de prueba	Origen	Prio.	Datos y pasos	Resultado esperado	Evidencia	Estado
CP-17	Mensaje vacío exacto por categoría sin equipos	H	P1	Categoría sin resultados · Filtrar una categoría vacía	Texto exacto **“No hay equipos disponibles en esta categoría”**	UI `CatalogPage.test.tsx`	Aprobado
CP-18	Categoría desconocida no rompe el catálogo	IA	P3	Catálogo cargado · `GET /api/equipment?category=Inexistente`	HTTP 200 y catálogo completo sin error	API `equipment.spec.ts`	Aprobado
CP-19	Endpoint de equipos disponibles	H	P2	Inventario con equipos prestados · `GET /api/equipment/available`	Solo equipos `AVAILABLE`; excluye `BORROWED`	API `equipment.spec.ts`	Aprobado
CP-20	Búsqueda por nombre o código de inventario	IA	P2	Catálogo cargado · Escribir “router” en el buscador	Se muestran solo coincidencias; se ocultan las demás tarjetas	UI `CatalogPage.test.tsx`	Aprobado
CP-21	Indicadores del catálogo	IA	P3	Datos sembrados · `GET /api/equipment/stats`	Totales de equipos, disponibles, prestados, pendientes y estudiantes correctos	API `equipment.spec.ts` · UI `CatalogPage.test.tsx`	Aprobado
CP-22	Detalle de equipo inexistente	IA	P3	Sesión activa · `GET /api/equipment/no-existe`	HTTP 404	API `equipment.spec.ts`	Aprobado
CP-23	Catálogo exige autenticación	H	P2	Sin token · `GET /api/equipment`	HTTP 401	API `equipment.spec.ts`	Aprobado

HU-03 · Reserva de equipos

ID	Caso de prueba	Origen	Prio.	Datos y pasos	Resultado esperado	Evidencia	Estado
CP-24	Registrar reserva pendiente	H	P1	Equipo disponible y estudiante activo · Equipo + fecha futura + 08:00–10:00	HTTP 201, estado `PENDING`, sin penalización	API `reservations.spec.ts` · E2E `05-reserva-registrada.png`	Aprobado
CP-25	Bloquear cruce con reserva aprobada del mismo equipo	H	P1	Reserva aprobada 09:00–11:00 · Solicitar 08:00–10:00 para el mismo equipo y fecha	HTTP 400 indicando cruce con reserva aprobada	API `reservations.spec.ts`	Aprobado
CP-26	Una sola reserva activa simultánea por estudiante	H	P1	Reserva pendiente 08:00–10:00 · Segunda reserva 09:00–11:00 en otro equipo	HTTP 400: el estudiante ya tiene una reserva activa	API `reservations.spec.ts`	Aprobado
CP-27	Reservas contiguas sin cruce son válidas	IA	P2	Reserva aprobada 08:00–10:00 · Solicitar 10:00–12:00	HTTP 201; los extremos contiguos no se consideran cruce	API `reservations.spec.ts`	Aprobado
CP-28	Equipo no disponible es rechazado	H	P1	Equipo en estado `BORROWED` · Intentar reservarlo	HTTP 400 “no está disponible”	API `reservations.spec.ts`	Aprobado
CP-29	Fecha pasada es rechazada	IA	P2	Sesión activa · Fecha `2020-01-01`	HTTP 400 por fechas pasadas	API `reservations.spec.ts`	Aprobado
CP-30	Hora de inicio posterior a la de fin	H	P1	Formulario abierto · Inicio 12:00, fin 10:00	HTTP 400; la UI muestra “anterior a la hora de fin”	API `reservations.spec.ts` · UI `NewReservationPage.test.tsx`	Aprobado
CP-31	Máximo 4 horas continuas	H	P1	Formulario abierto · 08:00–13:00	HTTP 400; la UI muestra límite de 4 horas	API `reservations.spec.ts` · UI `NewReservationPage.test.tsx`	Aprobado
CP-32	Horario fuera de atención	IA	P2	Formulario abierto · 06:00–08:00	HTTP 400; atención 07:00–22:00	API `reservations.spec.ts`	Aprobado
CP-33	Formato de hora inválido	IA	P2	Petición directa · `startTime: "8:00"`	HTTP 400 con errores de validación del DTO	API `reservations.spec.ts`	Aprobado
CP-34	Estudiante penalizado no puede reservar	H	P1	Usuario `PENALIZED` · Intentar crear reserva	HTTP 400 indicando cuenta penalizada	API `reservations.spec.ts`	Aprobado
CP-35	Administrador no puede crear reservas	H	P2	Sesión de administrador · `POST /api/reservations`	HTTP 403	API `reservations.spec.ts`	Aprobado
CP-36	Equipo inexistente	IA	P2	Sesión de estudiante · `equipmentId` inválido	HTTP 404 “no existe”	API `reservations.spec.ts`	Aprobado
CP-37	Estudiante solo ve sus propias reservas	H	P1	Reservas de dos estudiantes · `GET /api/reservations/me`	Solo las reservas del estudiante autenticado	API `reservations.spec.ts`	Aprobado
CP-38	Reserva exige autenticación	H	P2	Sin token · `POST /api/reservations`	HTTP 401	API `reservations.spec.ts`	Aprobado
CP-39	Preselección del equipo desde el catálogo	IA	P2	Sesión activa · Abrir `/reservar?equipo=<id>`	El selector y el panel de detalle muestran el equipo elegido	UI `NewReservationPage.test.tsx`	Aprobado
CP-40	Confirmación de solicitud registrada	H	P2	Equipo preseleccionado · Enviar formulario válido	Mensaje “Reserva registrada. Queda pendiente de aprobación.” y redirección a Mis reservas	UI `NewReservationPage.test.tsx` · E2E `05-reserva-registrada.png`	Aprobado
CP-41	Duración rápida de 1 a 4 horas	IA	P3	Formulario abierto · Clic en “2 h”	Hora de fin ajustada a inicio + 2 horas sin exceder 22:00	UI `NewReservationPage.tsx` (validación local)	Aprobado

HU-04 · Aprobación y rechazo de solicitudes

ID	Caso de prueba	Origen	Prio.	Datos y pasos	Resultado esperado	Evidencia	Estado
CP-42	Listar solicitudes pendientes por defecto	H	P1	Solicitudes en `PENDING` · `GET /api/admin/reservations`	Solo pendientes con datos de estudiante y equipo	API `admin.spec.ts` · UI `AdminPendingPage.test.tsx`	Aprobado
CP-43	Filtro por estado y opción ALL	IA	P2	Reservas en varios estados · `?status=APPROVED` y `?status=ALL`	Filtrado correcto y listado completo respectivamente	API `admin.spec.ts`	Aprobado
CP-44	Aprobar reserva y marcar equipo como prestado	H	P1	Solicitud pendiente · Clic en Aprobar	Estado `APPROVED`, revisor y fecha; equipo pasa a `BORROWED`; evento de auditoría	API `admin.spec.ts` · E2E `06-solicitud-aprobada.png`	Aprobado
CP-45	No aprobar una reserva que cruza otra aprobada	H	P1	Reserva aprobada 09:00–11:00 · Aprobar pendiente 10:00–12:00	HTTP 400 por cruce con otra reserva aprobada	API `admin.spec.ts`	Aprobado
CP-46	Solo se aprueban reservas pendientes	IA	P2	Reserva ya aprobada · Volver a aprobar	HTTP 400 indicando que solo aplica a pendientes	API `admin.spec.ts`	Aprobado
CP-47	Rechazo exige motivo obligatorio	H	P1	Solicitud pendiente · Abrir modal y confirmar sin motivo	Error “al menos 5 caracteres”; no se llama al API	API `admin.spec.ts` · UI `AdminPendingPage.test.tsx`	Aprobado
CP-48	Motivo de rechazo demasiado corto	IA	P2	Solicitud pendiente · Motivo `abc`	HTTP 400 y la reserva permanece `PENDING`	API `admin.spec.ts`	Aprobado
CP-49	Rechazo guarda el motivo visible al estudiante	H	P1	Solicitud pendiente · Motivo “Equipo en mantenimiento preventivo”	Estado `REJECTED`, motivo y revisor persistidos; visible en Mis reservas	API `admin.spec.ts` · UI `MyReservationsPage.test.tsx`	Aprobado
CP-50	Estudiante sin acceso al panel administrativo	H	P1	Sesión de estudiante · Consultar reservas y métricas administrativas	HTTP 403 en todos los endpoints	API `admin.spec.ts`	Aprobado
CP-51	Métricas administrativas	IA	P2	Datos sembrados · `GET /api/admin/stats`	Pendientes, aprobadas, devoluciones de hoy, penalizados e inventario correctos	API `admin.spec.ts` · UI `AdminPendingPage.test.tsx`	Aprobado
CP-52	Bitácora de auditoría disponible	H	P2	Eventos registrados · `GET /api/admin/audit?limit=10`	Eventos con tipo, actor, fecha e IP	API `admin.spec.ts`	Aprobado
CP-53	Aprobar reserva inexistente	IA	P3	Sesión de administrador · `PATCH /api/admin/reservations/no-existe/approve`	HTTP 404	API `admin.spec.ts`	Aprobado

HU-05 · Devolución y penalización automática

ID	Caso de prueba	Origen	Prio.	Datos y pasos	Resultado esperado	Evidencia	Estado
CP-54	Devolución puntual sin penalización	H	P1	Reserva aprobada vigente · Registrar devolución antes del fin	`late=false`, `penaltyApplied=false`; estudiante sigue `ACTIVE`	API `admin.spec.ts`	Aprobado
CP-55	Devolución tardía penaliza automáticamente	H	P1	Reserva aprobada con fecha vencida · Registrar devolución	`late=true`, `penaltyApplied=true`; estudiante pasa a `PENALIZED`	API `admin.spec.ts` · E2E `07-devolucion-penalizada.png`	Aprobado
CP-56	El equipo vuelve a estar disponible	H	P1	Cualquier devolución · Confirmar devolución	Equipo en estado `AVAILABLE`	API `admin.spec.ts`	Aprobado
CP-57	Solo se devuelven reservas aprobadas	IA	P2	Reserva pendiente · Intentar devolución	HTTP 400 indicando que solo aplica a aprobadas	API `admin.spec.ts`	Aprobado
CP-58	Evento de penalización en la bitácora	H	P1	Devolución tardía registrada · Consultar bitácora	Evento `PENALTY_APPLIED` con el código del estudiante	API `admin.spec.ts` · E2E `08-bitacora-penalizacion.png`	Aprobado
CP-59	Aviso de cuenta penalizada al estudiante	IA	P2	Usuario `PENALIZED` · Abrir Mis reservas	Banner “Tu cuenta está penalizada” visible	UI `MyReservationsPage.test.tsx`	Aprobado
CP-60	Advertencia previa a confirmar devolución tardía	IA	P2	Préstamo vencido · Abrir modal de devolución	Mensaje que anuncia el cambio automático a Penalizado	UI `AdminReturnsPage.test.tsx` · E2E	Aprobado
CP-61	Indicador de puntualidad del préstamo	H	P3	Préstamo con fecha vencida · Abrir panel de devoluciones	Etiqueta “Vencida hace X h Y min”	UI `AdminReturnsPage.test.tsx`	Aprobado

Anexo A.2. Pruebas de rendimiento

ID	Transacción	Origen	Prio.	Umbral	Resultado medido	Estado
PC-01	`POST /api/auth/login` (JWT)	H	P1	p95 < 3 000 ms	90 ms p95 · 76.1 ms promedio · 0 errores	Aprobado
PC-02	`GET /api/equipment`	H	P1	p95 < 3 000 ms	18 ms p95 · 14.1 ms promedio · 0 errores	Aprobado
PC-03	`GET /api/equipment?category=Cómputo`	H	P2	p95 < 3 000 ms	18 ms p95 · 14.1 ms promedio · 0 errores	Aprobado
PC-04	`GET /api/equipment/stats`	IA	P2	p95 < 3 000 ms	18 ms p95 · 14.2 ms promedio · 0 errores	Aprobado
PC-05	`GET /api/reservations/me`	IA	P2	p95 < 3 000 ms	18 ms p95 · 14.0 ms promedio · 0 errores	Aprobado

Anexo B. Índice de evidencias

Evidencia	Ubicación	Contenido
Cobertura backend	backend/coverage/index.html	Reporte HTML e LCOV de las 49 pruebas de API.
Cobertura frontend	frontend/coverage/index.html	Reporte HTML e LCOV de las 44 pruebas de UI.
Reporte E2E JUnit	frontend/e2e/reports/junit.xml	Resultado de los 7 flujos Selenium.
Capturas E2E	frontend/e2e/screenshots/	Ocho capturas de los flujos críticos.
Datos crudos JMeter	jmeter/reports/results.jtl	194 629 muestras de la prueba de carga.
Reporte HTML JMeter	jmeter/reports/html/index.html	Dashboard interactivo de rendimiento.
Resumen JMeter	jmeter/reports/summary.md	Métricas, tablas y veredicto.
Gráficas JMeter	jmeter/reports/charts/	Tiempo de respuesta, percentiles y throughput.
Matriz de pruebas	docs/matriz-pruebas.md	61 casos funcionales y 5 de rendimiento.
Configuración Sonar	sonar-project.properties	Fuentes, exclusiones y cobertura.
Datos SonarQube	docs/sonar-data.json	Métricas e issues antes y después.
Backlog	docs/refactoring-backlog.md	Deuda técnica priorizada y estado.

Tabla B1. Índice de artefactos de evidencia del proyecto.

Anexo C. Reproducción de la evaluación

Los siguientes comandos permiten reproducir la totalidad de la evidencia desde una instalación limpia. Se ejecutan desde la raíz del repositorio.

```
# 1. Base de datos y datos demo
cd backend
npm install
npm run db:setup

# 2. API y frontend
npm run build && npm start
cd ..\frontend
npm install && npm run build && npm run preview

# 3. Pruebas
cd ..\backend
npm run test:cov
cd ..\frontend
npm run test:cov
npm run test:e2e

# 4. Carga
cd ..\jmeter
powershell -ExecutionPolicy Bypass -File .\run-load-test.ps1

# 5. Análisis estático
powershell -ExecutionPolicy Bypass -File ..\scripts\start-sonarqube.ps1
powershell -ExecutionPolicy Bypass -File ..\scripts\scan-sonar.ps1
```

Credenciales de demostración: estudiante 20240001 / Estudiante2026! y administrador ADMIN01 / Admin2026!. Después de una ejecución E2E se debe ejecutar npm run db:reset para restaurar el seed determinista.